2026

Алгоритмизация и программирование (2 семестр)

МГТУ им. Н.Э. Баумана
Факультет Информатика и системы управления
Кафедра Компьютерные системы и сети
Лектор: д.т.н., проф.
Иванова Галина Сергеевна

Дисциплина Алгоритмизация и программирование

Объем дисциплины во 2-м семестре – 5 зачетных единиц – 180 час.:

- лекции – 51 час - знакомство с теоретическим материалом;
- семинары – 17 часов - разработка алгоритмов решения задач;
- лабораторные работы – 34 часа – 8*4+2(зачет) часов - изучение приемов программирования;
- самостоятельная работа – 16*4 часа - закрепление материала (4 часа в неделю).

Содержание дисциплины:

Модуль 1 (8 неделя). C++ Библиотеки классов и более сложные элементы объектной модели.

Модуль 2 (17 неделя). C# Второй объектно-ориентированный язык программирования.

Языки программирования: **C++, C#**

Среды программирования: **Qt Creator, MonoDevelop**

Операционная система: **ASTRA Linux**

Отчетность по дисциплине :

- 2 рубежных контроля (РК):

- РК 1. C++. Создание иерархии классов с интерфейсом Qt - 2 часа – 7-8 недели.
- РК 2. C# Создание иерархии классов - 2 часа – 15-16 недели.
- экзамен.

Экзамен можно сдать досрочно до 10 мая, если сдать все лабораторные и ДЗ.

Проектно-технологическая практика

Группы	Объем практики	Задание 1, баллов	Задание 2, баллов	Задание 3, баллов	Задание 4, баллов
ИУ6-21Б, ИУ6-22Б, ИУ6-23Б, ИУ6И-21Б, ИУ6Ц-42Б	2 з.е.	21..35	не выпоняется	15-25	24..40
ИУ6-24Б, ИУ6-25Б	3 з.е.	15..25	15..25	12..20	18..30

1-2 Системы с графическим интерфейсом на C++.

3. Средства создания гр. интерфейсов на C#.

4. Система с графическим интерфейсом на C#.

До 25 февраля необходимо подготовить 2 экземпляра бланка задания (бланк печатается с двух сторон) и подписать самим и у преподавателя, ведущего практику, затем второй экземпляр отдать мне:

- 1-й экз. пойдет в отчет по практике,
- 2-й экз. останется на кафедре.

Контроль знаний по практике – дифференцированный зачет по баллам:

60..70 – удовлетворительно;

71..84 – хорошо;

85..100 – отлично

Учебные материалы

1. Иванова Г.С., Ничушкина Т.Н. Объектно-ориентированное программирование. Учеб. для вузов. – М.: Изд-во МГТУ им. Н.Э. Баумана, 2014. 456 с.
2. Павловская Т.А. C/C++. Программирование на языке высокого уровня: учебник для вузов. СПб.: Питер, 2021. 464 с.: ил. URL: https://ibooks.ru/bookshelf/376844.
3. Программирование на языке высокого уровня C# Учебное пособие / Павловская Т.А. - 2021. - URL: http://www.iprbookshop.ru/102051.html.
4. Шилдт Г. Полный справочник по C++, 4 изд. М.: Изд. дом "Вильямс", 2015. 800с.
5. Шлее М. Qt4. 5. Профессиональное программирование на C++ / Шлее М. - СПб. : БХВ-Петербург, 2018. - 884 с. + CD. - (В подлиннике). - ISBN 978-5-9775-0398-3.
6. Уроки C++. URL: https://ravesli.com/uroki-cpp/ .
7. Сайт кафедры: https://e-learning.bmstu.ru/iu6/course/view.php?id=366

Консультации д.т.н., проф. Г.С. Ивановой

а) на кафедре ИУ6 (главное здание, 8 этаж, ауд. 807):

- понедельник с 13-30 до 15-00;
- вторник числ. с 14-00 до 16-00
  знам. с 12-00 до 14-00

б) по электронной почте:

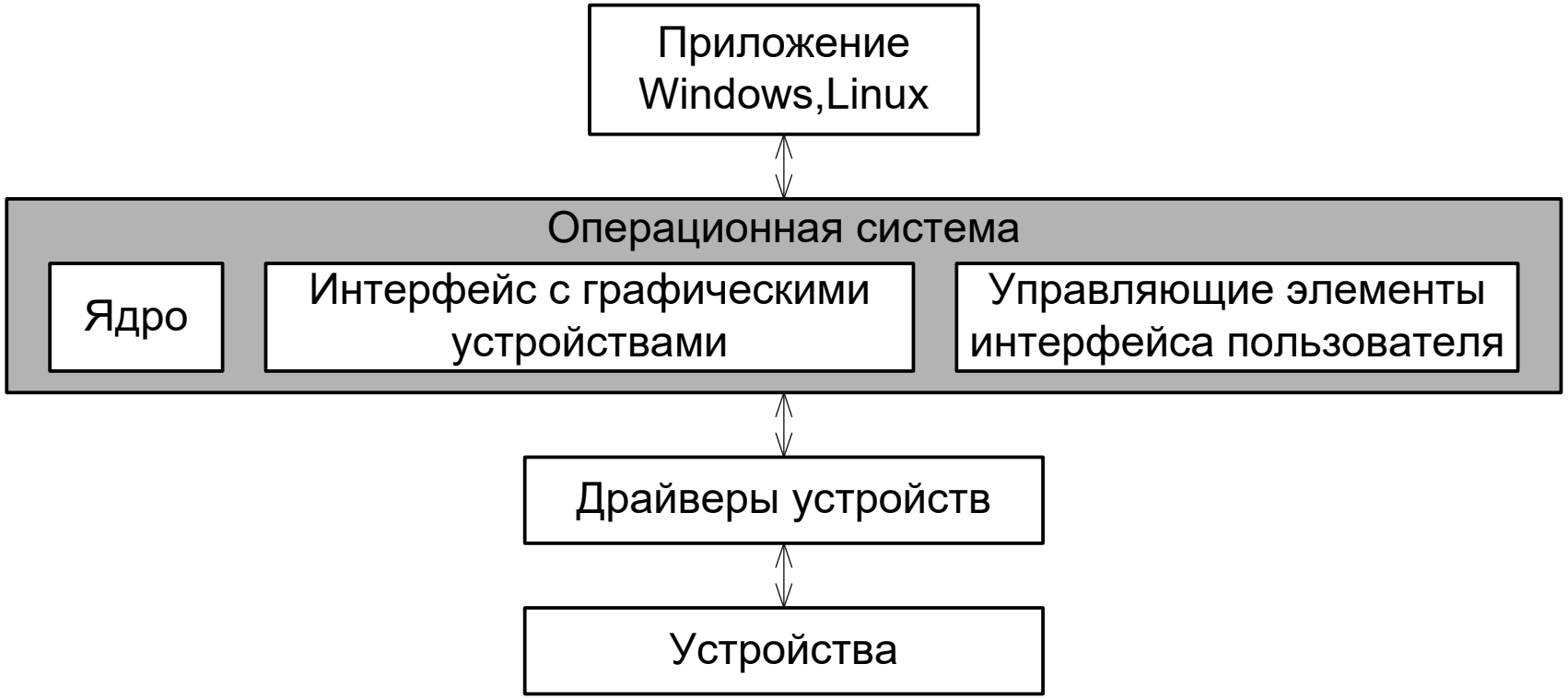
gsivanova@gmail.com,
gsivanova@bmstu.ru

АИП 2026

Глава 9. Создание приложений с графическими интерфейсами. Объектная библиотека Qt

МГТУ им. Н.Э. Баумана
Факультет Информатика и системы управления
Кафедра Компьютерные системы и сети
Лектор: д.т.н., проф.
Иванова Галина Сергеевна

Введение. Особенности создания программ с графическим интерфейсом



Управление техническими средствами осуществляется через *Application Program Interface* (API) – набор из нескольких тысяч функций, выполняющих все системно-зависимые действия, такие как выделение памяти, вывод на экран и т.д.

Принцип событийного управления

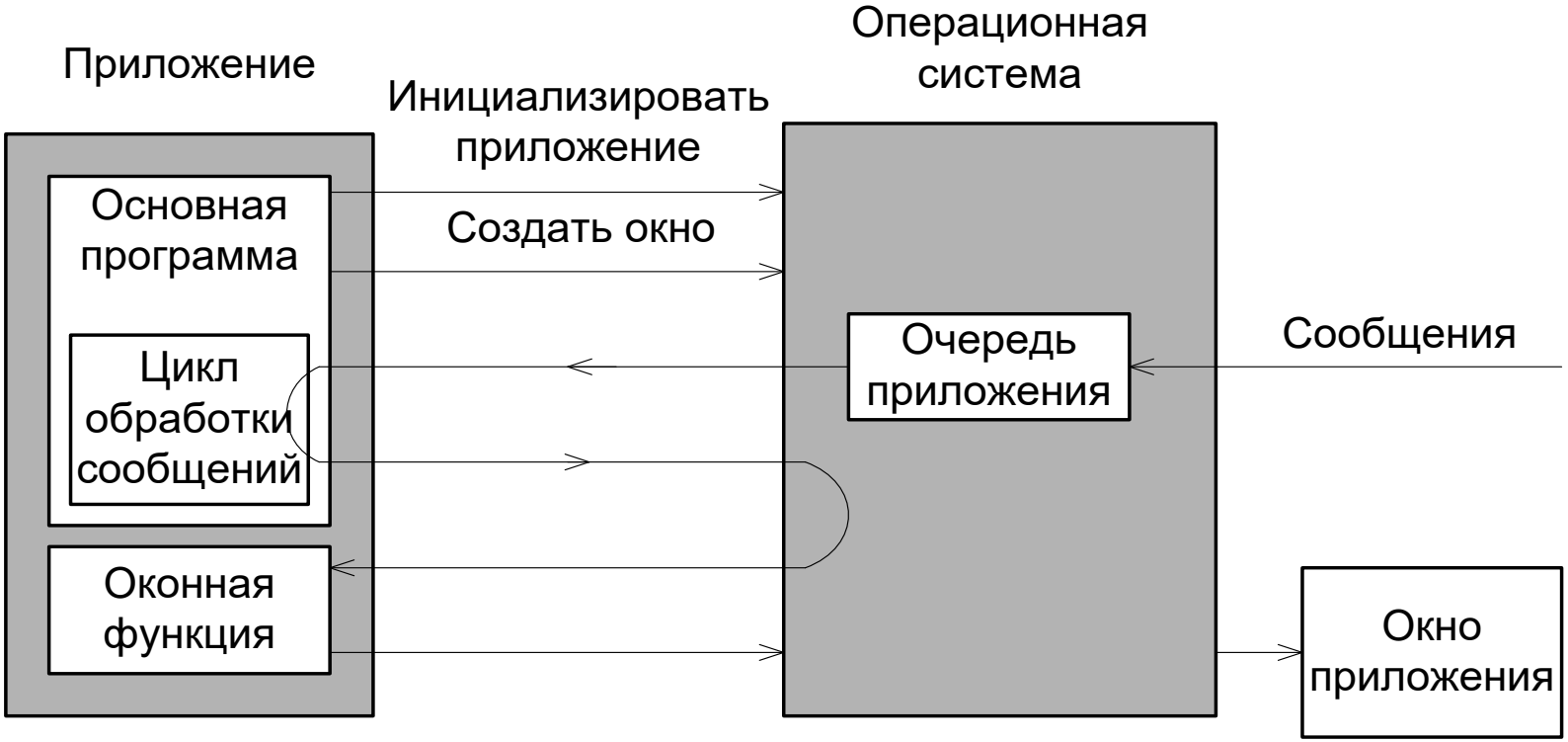
Приложение (в отличие от программы) – набор подпрограмм, вызываемых при наступлении некоторого *события*, которым считается любое изменение в системе, касающееся данного приложения.

Каждому приложению на экране соответствует окно.

Окно – самостоятельно существующий объект, параметры которого хранятся в специальной структуре данных, а поведение определяется обработчиками сообщений, составляющими ***оконную функцию***.

Структура приложения

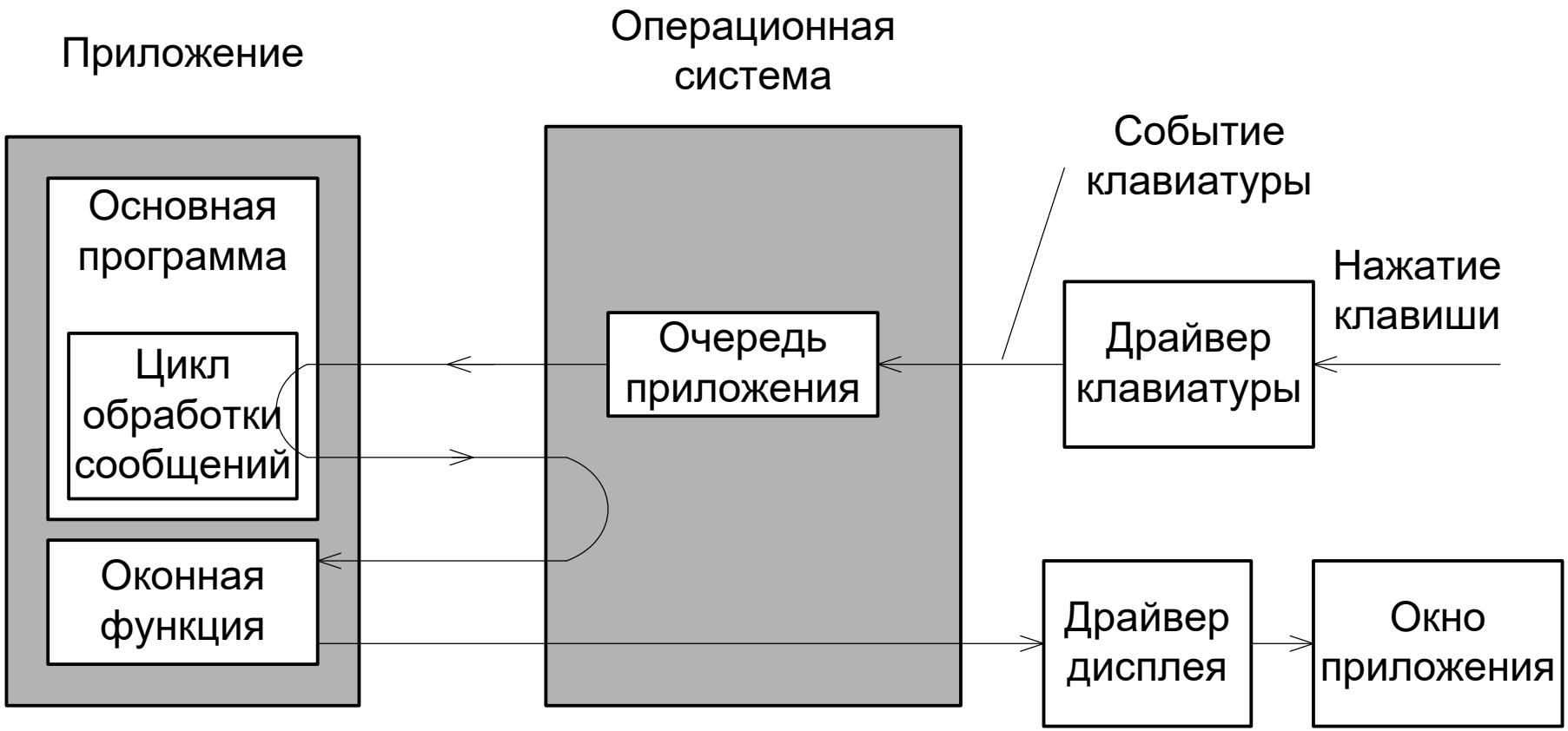
Минимально любое приложение с графическим интерфейсов состоит из двух частей: *основной программы* и *оконной функции*.



Появившиеся в очереди сообщения выбираются циклом обработки сообщений и передаются *через операционную систему* соответствующей оконной функции приложения.

Для выполнения действий обработчики сообщений обращаются к функциям API.

Обработка сообщения от клавиатуры



9.1 Классы окон (формы). Класс QWidget

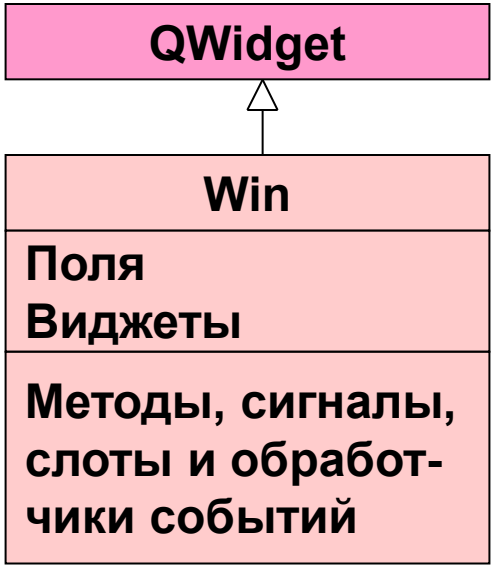
Каждому элементу оконного интерфейса соответствует *виджет* – объект интерфейсного класса библиотеки Qt.

Виджеты визуальных компонентов управляются контейнером – главным виджетом – виджетом управления окном приложения. В качестве такого виджета могут использоваться объекты родоначальника всех виджетов - класса **`QWidget`** или специальных классов для окон: **`QMainWindow`** и **`QDialog`**.

Объект класса **`QWidget`** или его потомков, не встроенные в родительский виджет, становится окном приложения (формой).

Используя классы **`QWidget`**, **`QMainWindow`** и **`QDialog`** в качестве базовых классов, можно построить класс объекта-окна или формы.

Классы форм кроме обычных полей, полей-виджетов и обычных методов могут включать методы-сигналы, методы-слоты (обработчики сигналов) и методы - обработчики событий.



Класс QWidget

Простое окно приложения может строиться на базе класса **`QWidget`**.

Объекта класса **`QWidget`**, не встроенный в родительский виджет, становится окном приложения (формой).

Основные свойства:

`windowTitle` – заголовок окна;

`windowIcom` – пиктограмма приложения;

`width, height` – размеры;

`font` – характеристики шрифта;

`cursor` – форма курсора мыши и т. д.

Некоторые методы:

конструктор, деструктор

`setWindowTitle()` – задать заголовок;

`setWindowIcon()` – задать пиктограмму

Некоторые слоты:

`show()` – показать;

`hide()` – спрятать;

`close()` – закрыть.

События, генерируемые объектами класса QWidget

а) ***при изменении состояния формы:***

`create()` – при создании формы;

`destroy()` – при уничтожении формы;

`showEvent()` – перед визуализацией формы;

`hideEvent()` – перед удалением формы с экрана;

`enterEvent()` – курсор мыши появляется над виджетом;

`leaveEvent()` – когда курсор мыши покидает форму;

`paintEvent()` – при необходимости нарисовать или перерисовать форму;

`resizeEvent()` - при изменении размеров формы на экране;

`closeEvent()` – при закрытии формы.

События, генерируемые объектами класса QWidget (2)

б) от клавиатуры и мыши:

`keyPressEvent()` – при нажатии клавиш клавиатуры;

`keyReleaseEvent()` – при отпускании клавиш клавиатуры;

`mousePressEvent()` – при нажатии клавиш мыши;

`mouseDoubleClickEvent()` – при двойном нажатии клавиш мыши

`mouseReleaseEvent()` – при отпускании клавиш мыши;

`mouseMoveEvent()` – при перемещении мыши (многократно);

в) при перетаскивании объекта мышью:

`dragEnterEvent()` – в момент начала перетаскивания объекта мышью над формой;

`dragLeaveEvent()` – в момент опускания объекта на форму;

`dragMoveEvent()` – в процессе перетаскивания объекта над формой (многократно);

г) другие…

9.2 Основные виджеты

1. *Метка (класс QLabel)*

Метка – окно с текстом. Может использоваться для формирования на форме надписей или подписей.

TextLabel

Основные свойства:

`text` – заголовок – текст, выводимый в окне компонента.

`objectName` – имя компонента в программе.

`alignment` – определяет способ выравнивания текста в окне виджета по горизонтали и вертикали, например: *AlignHCenter* - по центру; *AlignLeft* - по левой границе; *AlignRight* - по правой границе.

`font` – определяет шрифт текста.

`palette` – определяет цвета текста и фона в окне.

`wordWrap` – определяет, разбивать или нет текст на строки.

Основной метод:

`SetVisible()` – параметр *true* – видна, *false* – скрыта.

Основные визуальные компоненты (2)

2. Строчный редактор (класс QLineEdit)

Виджет представляет собой окно, обычно выделенное цветом, которое может использоваться, например, для организации ввода информации.

Основные свойства:

`text` – вводимая/выводимая строка текста;

`readOnly` **–** определяет возможность ввода информации в окно;

`objectName` **–** имя виджета в программе.

Основные методы:

`clear()` **–** очистка поля text;

`setFocus()` – установка фокуса ввода;

`setVisible()` **-** параметр *true* – виден, *false* – скрыт;

`setEnable()` – **-** параметр *true* – доступен, *false* – недоступен.

Основные события:

`returnPressed()` – нажатие Enter (*при наличии QValidator).

Основные визуальные компоненты (3)

3. Кнопка (QPushButton)

Виджет представляет собой окно, в котором размещается название кнопки. Используется для инициирования каких-либо действий.

Основные свойства:

`text` - название кнопки.

`objectName` – имя виджета в программе.

`default` – определяет, генерируется ли событие *clicked()* для данной кнопки при нажатии клавиши *Enter*.

Основные события:

`clicked()` – «щелчок» мышкой в зоне виджета.

Основные методы:

`SetFocus()` – установка фокуса ввода.

`setVisible()` – установка видимости виджета.

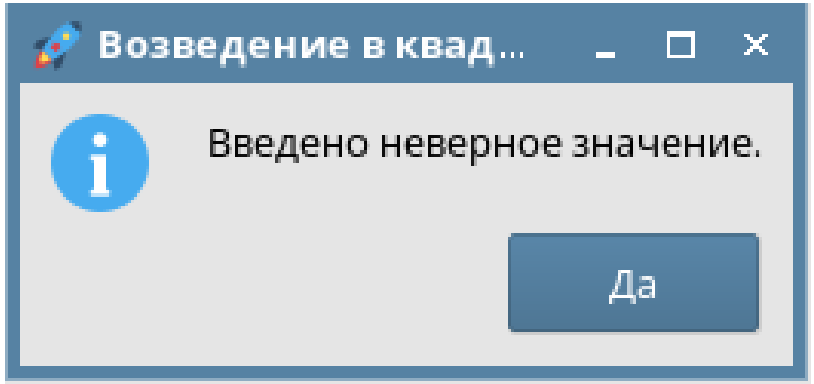
`setEnable()` – установка доступности виджета.

Визуализация сообщений пользователю

Класс `QMessageBox`

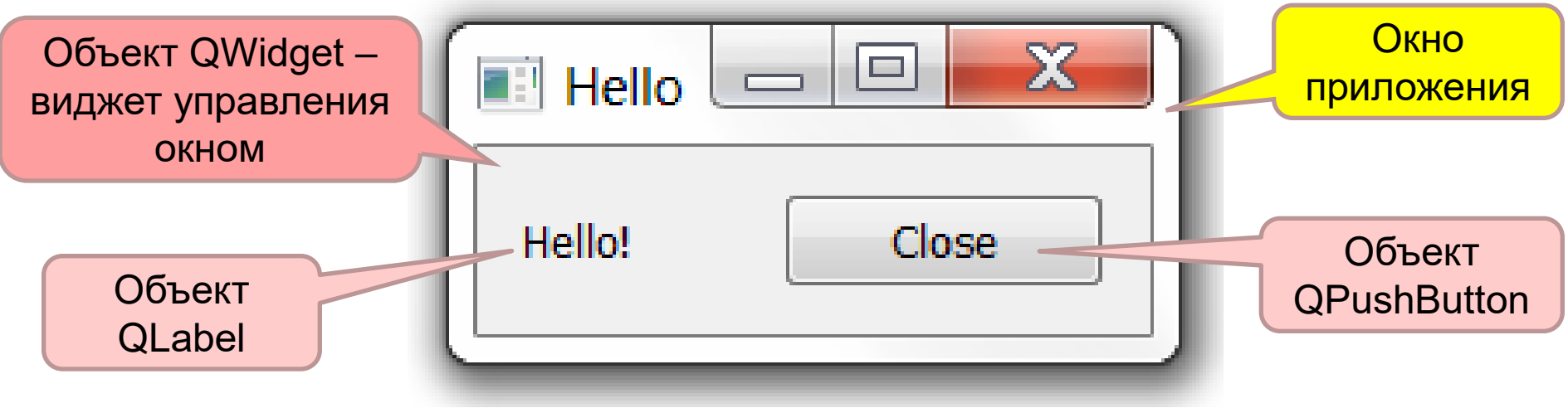
Пример:

```
QMessageBox msgBox(QMessageBox::Information,
    codec->toUnicode("Возведение в квадрат."),
    codec->toUnicode("Введено неверное значение."),
    QMessageBox::Ok);
msgBox.exec();
```



9.3 Простейшая программа с Qt интерфейсом

Пример 9.1 Приложение Hello



Текст программы

```
#include <QApplication>
#include <QLabel>
#include <QPushButton>
#include <QHBoxLayout>
int main(int argc,char *argv[]) {
    QApplication app(argc,argv);
    QWidget win;
    win.setWindowTitle("Hello");
    QLabel *helloLabel=new QLabel("Hello!",&win);
    QPushButton *exitButton=
                    new QPushButton("Close",&win);
    QHBoxLayout *layout = new QHBoxLayout(&win);
    layout->addWidget(helloLabel);
    layout->addWidget(exitButton);

    QObject::connect(exitButton,SIGNAL(clicked(bool)),
                     &win,SLOT(close()));
    win.show();
    return app.exec();
}
```

Объект-приложение

Объект управления окном

Метка

Кнопка

Компоновщик

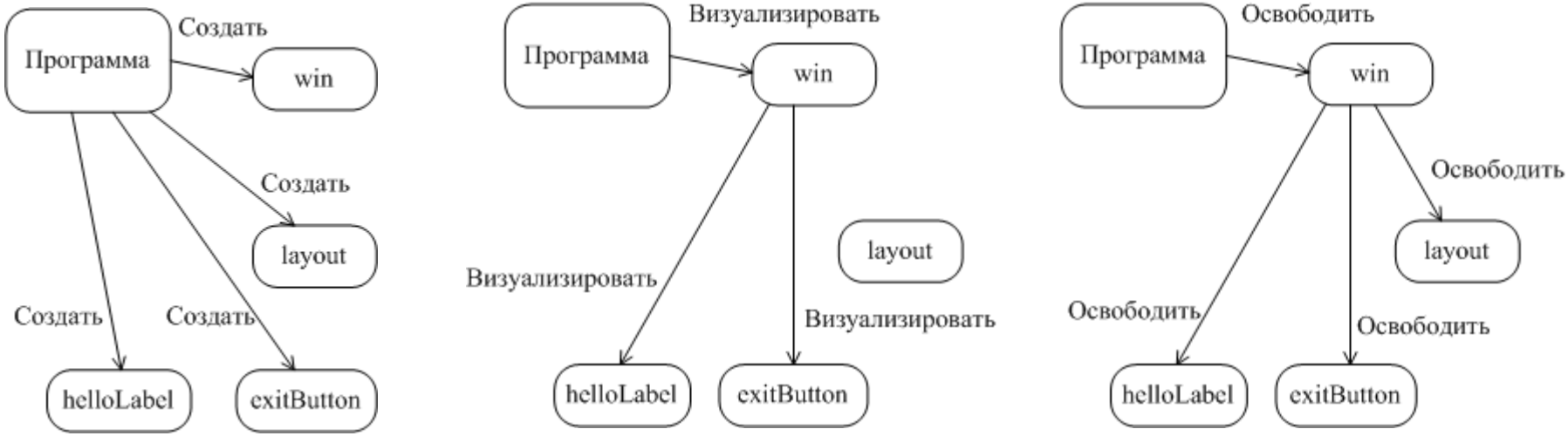
Связь сигнала со слотом

Визуализация окна

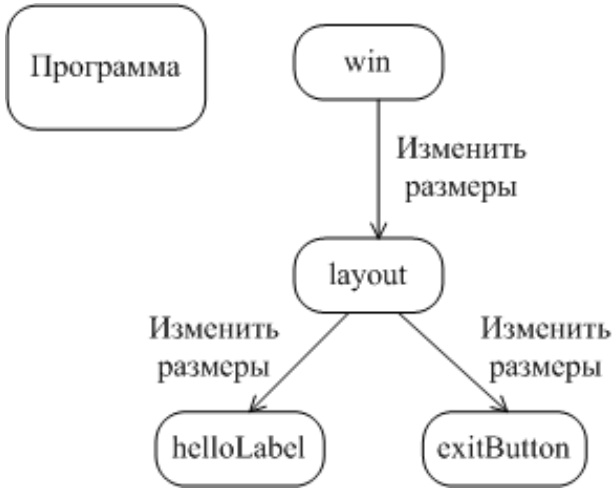
Запуск цикла обработки сообщений

Контейнерные свойства виджетов

Объект класса QWidget – win – контейнер, который отвечает за визуализацию компонентов и освобождение ими памяти.



Компоновщик – контейнер, который отвечает за политику изменения размеров визуальных компонентов.



9.4 Создание класса окна

Вид окна интерфейса

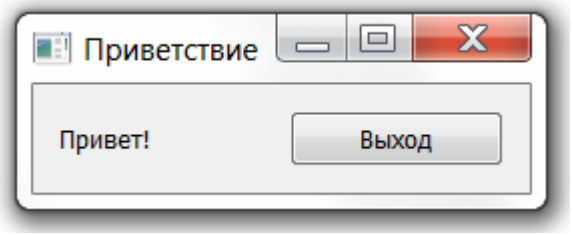
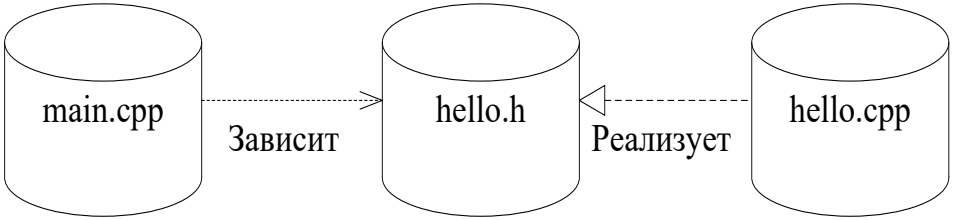


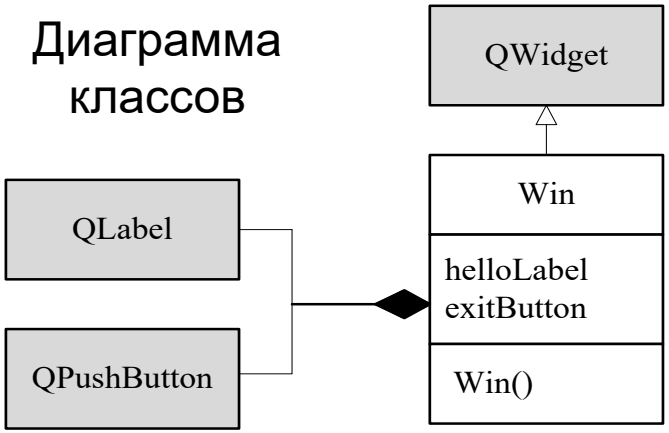
Диаграмма компоновки приложения



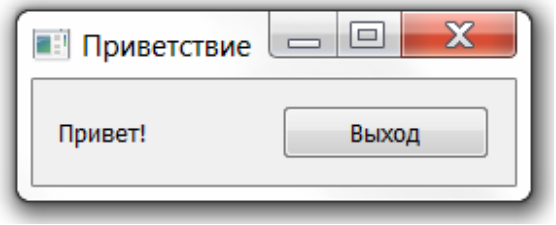
Пример 9.2. Файл hello.h:

```
#ifndef hello_h
#define hello_h
#include <QWidget>
#include <QLabel>
#include <QPushButton>
class Win: public QWidget
{
    QLabel *helloLabel;
    QPushButton *exitButton;
public:
    Win(QWidget *parent = nullptr);
};
#endif
```

Диаграмма классов



Конструктор класса окна и русификация интерфейса



Файл hello.cpp:

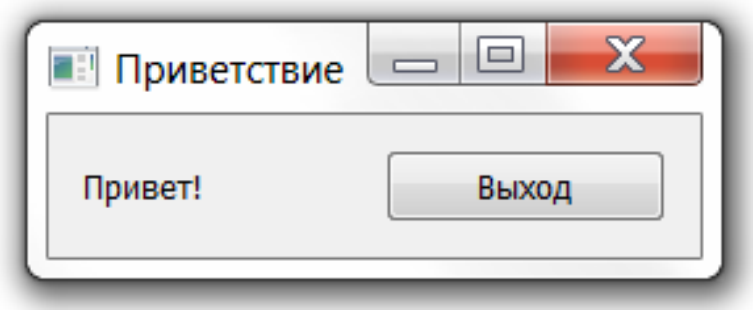
```
#include "hello.h"
#include <QTextCodec>
#include <QHBoxLayout>
Win::Win(QWidget *parent):QWidget(parent)
{
    QTextCodec *codec =
                QTextCodec::codecForName("Windows-1251");
    setWindowTitle(codec->toUnicode("Приветствие"));
    helloLabel = new QLabel(codec->toUnicode("Привет!"),this);
    exitButton =
         new QPushButton(codec->toUnicode("Выход"),this);
    QHBoxLayout *layout = new QHBoxLayout(this);
    layout->addWidget(helloLabel);
    layout->addWidget(exitButton);
    connect(exitButton,SIGNAL(clicked(bool)),
            this,SLOT(close()));
}
```

Перекодировщик для Windows, для Qt 6 не применяется

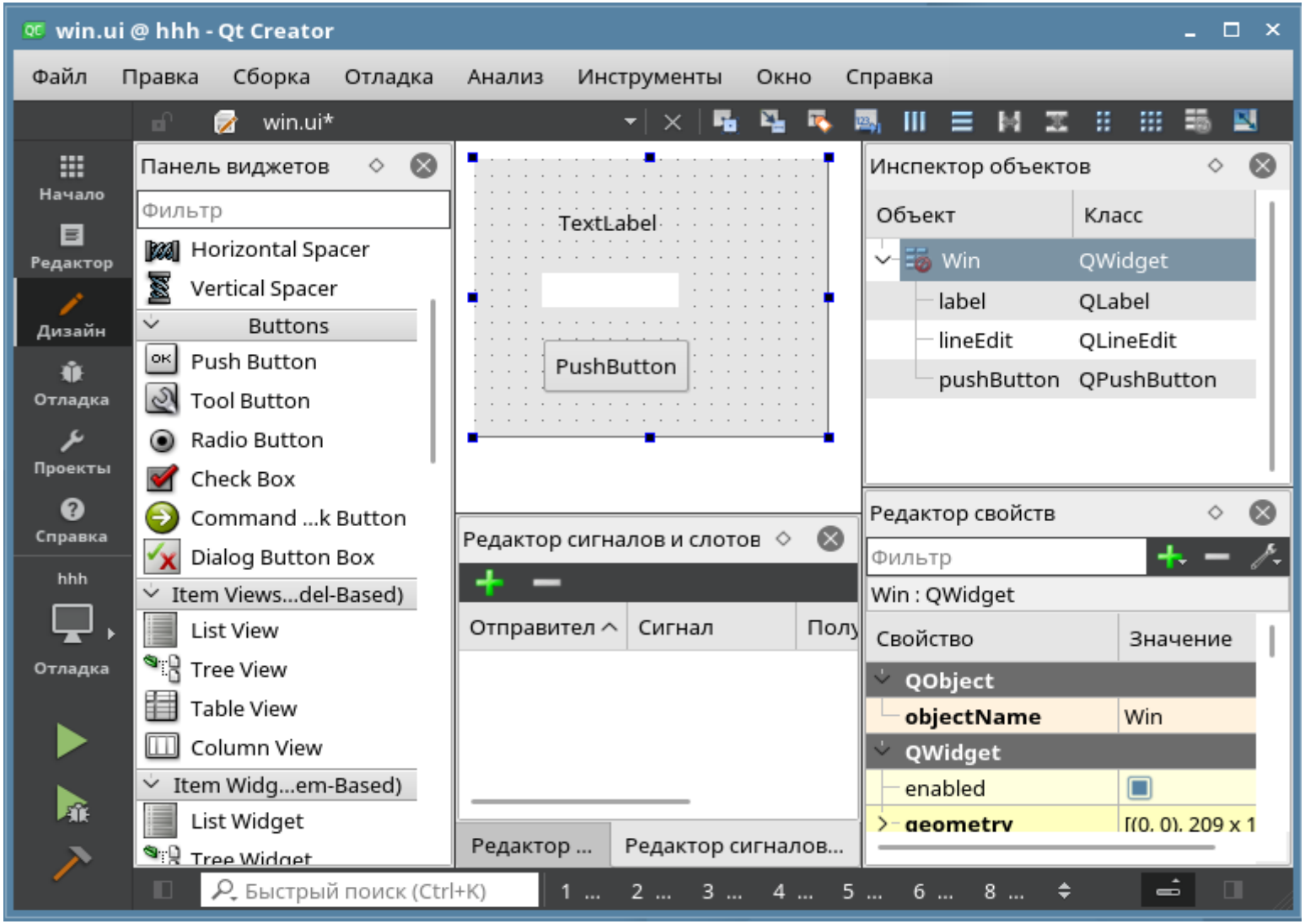
Основная программа

Файл main.cpp:

```
#include "hello.h"
#include <QApplication>

int main(int argc, char *argv[])
{
      QApplication app(argc, argv);
      Win win;
      win.show();
      return app.exec();
}
```



Визуальная разработка интерфейса с использованием Дизайнера



https://itproger.com/course/qt-creator 13 видео уроков

Класс QString

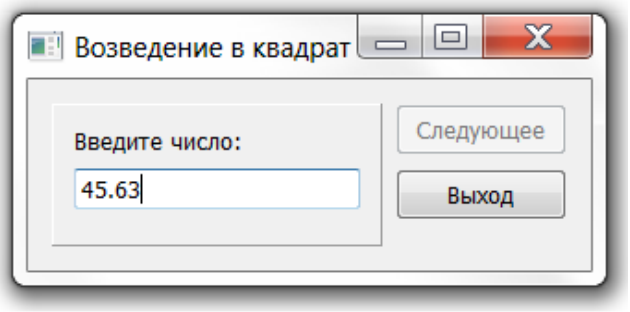
При работе с библиотекой Qt для представления текстовой информации обычно используется класс строки QString – строка символов Unicode.

Примеры:

```
QString str = "Hello";          // создание и инициализация строки
str[i] = QChar('U');            // изменение символа
if (str.at(i) >= QChar('a')… // доступ только для чтения
if (str == "auto")…             // сравнение строк
str = "and";
str.prepend("rock ");       // вставить перед str == "rock and"
str.append(" roll");        // вставить после str == "rock and roll"
str.replace(5, 3, "&");     // вставить вместо str == "rock & roll"
str.setNum(1234);           // число в строку str == "1234"
str.setNum(1.0/3,'f');      // число в строку str == "0.3333"
QString str = "FF"; bool ok;
int hex = str.toInt(&ok, 16);   // hex == 255, ok == true
QString str = "1234.56";
double val = str.toDouble();    // val == 1234.56
d = QString( "1234.56e-02" ).toDouble(&ok);
                                // ok == true, d == 12.3456
```

9.5 Механизм сигналов и слотов

Пример 9.3 Возведение числа в квадрат. Объявление новых слотов.



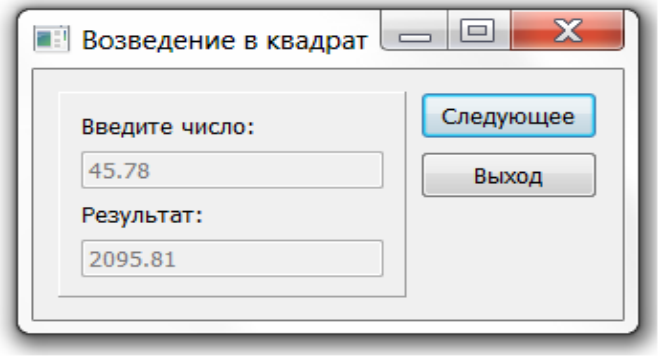


Диаграмма состояний интерфейса

Нормальное состояние

Нажатие кнопки Следующее/
Восстановление исходного состояния

Запуск приложения/
Инициализация
исходного состояния

Нажатие клавиши Enter
[Значение допустимо]/
Вычисление

Ввод значения

Вывод результата

Нажатие кнопки OK/
Возврат в состояние
ввода

Нажатие клавиши Enter
[Значение недопустимо]/
Обработка ошибки

Нажатие кнопки Выход/
Завершение работы

Сообщение об ошибке

Проектирование класса окна. Файл win.h.

Форма интерфейса

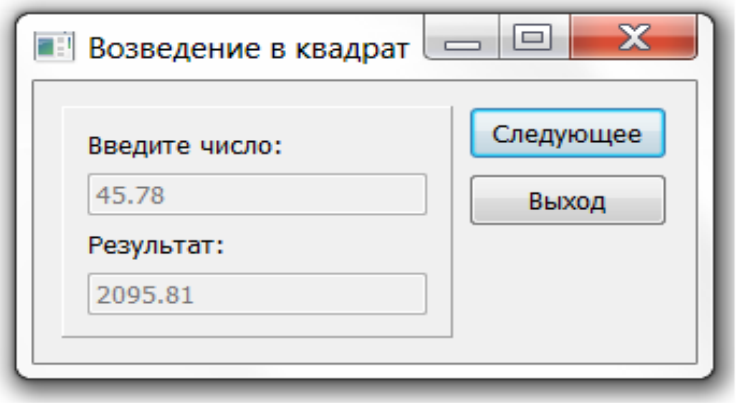


Диаграмма классов

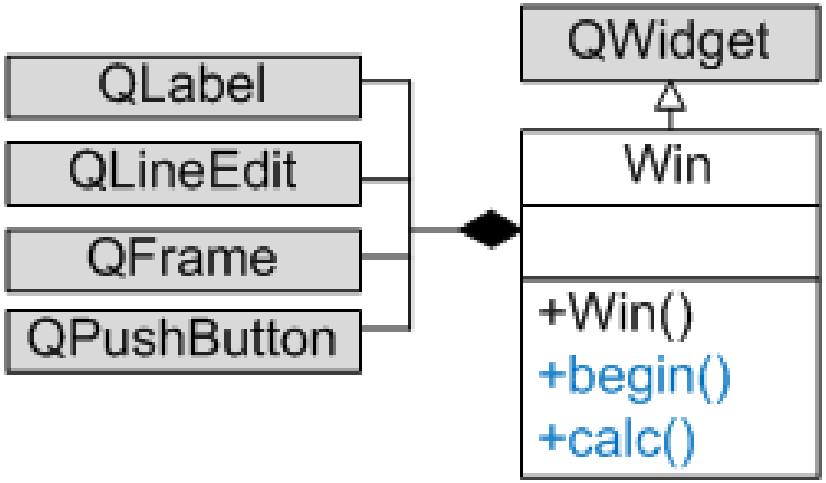
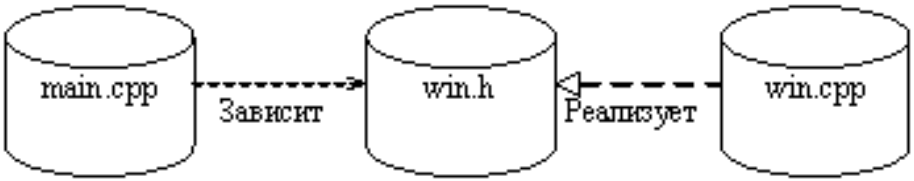
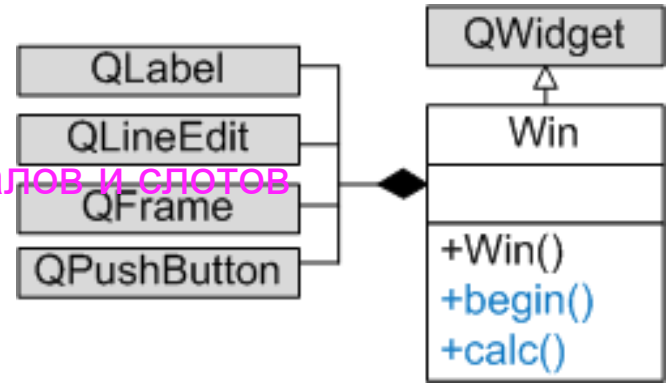


Диаграмма компоновки

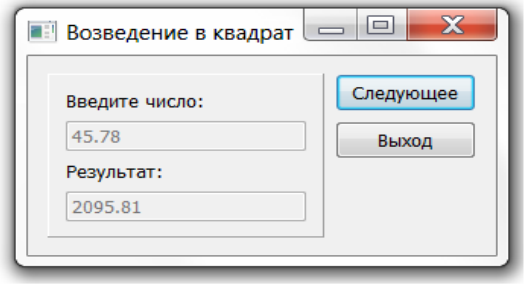



```
#ifndef win_h
#define win_h

#include <QWidget>
#include <QFrame>
#include <QLabel>
#include <QLineEdit>
#include <QPushButton>
#include <QValidator>
```

Файл win.h. Описание класса окна




```
class Win:public QWidget     // класс окна
{
    Q_OBJECT    // макрос Qt для реализации сигналов и слотов
protected:
    QTextCodec *codec;      // перекодировщик
    QFrame *frame;          // рамка
    QLabel *inputLabel;     // метка ввода
    QLineEdit *inputEdit;   // строчный редактор ввода
    QLabel *outputLabel;    // метка вывода
    QLineEdit *outputEdit;  // строчный редактор вывода
    QPushButton *nextButton; // кнопка Следующее
    QPushButton *exitButton; // кнопка Выход
public:
    Win(QWidget *parent = nullptr); // конструктс
public slots:
    void begin();           // инициализация интерфейса
    void calc();            // реализация вычислений
};
```



Файл win.h. Описание класса валидатора

```
class StrValidator:public QValidator // класс проверки ввода
{
public:
    StrValidator(QObject *parent):QValidator(parent){}
    virtual State validate(QString &str,int &pos)const;
    {
        return Acceptable;  // метод всегда принимает
                            // вводимую строку
    }
};
#endif
```

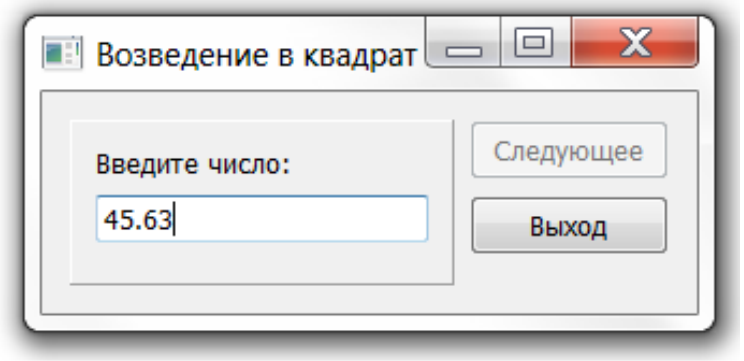
Объект-валидатор при создании связывается с объектом строчного редактора. Метод validate() автоматически вызывается для проверки вводимой строки. Если метод возвращает **`Acceptable`**, то редактор генерирует сигналы:

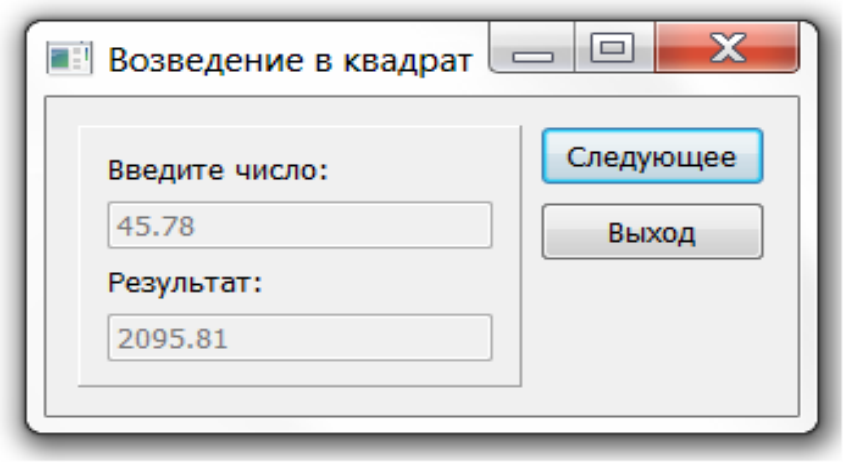
`editingFinished()` – завершение редактирования;

`returnPressed()` – нажатие клавиши Enter.

Файл win.cpp. Создание и настройка виджетов

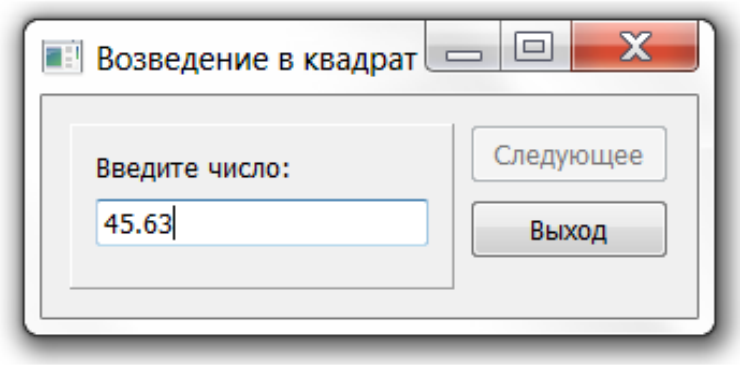
```
#include "win.h"
#include <QTextCodec>
#include <QVBoxLayout>
Win::Win(QWidget *parent):QWidget(parent)
{
    codec = QTextCodec::codecForName("Windows-1251");
    setWindowTitle(codec->toUnicode("Возведение в квадрат"));
    frame = new QFrame(this);
    frame -> setFrameShadow(QFrame::Raised);
    frame -> setFrameShape(QFrame::Panel);
    inputLabel =
          new QLabel(codec->toUnicode("Введите число:"),this);
    inputEdit = new QLineEdit("",this);
    StrValidator *v=new StrValidator(inputEdit);
    inputEdit -> setValidator(v);
```

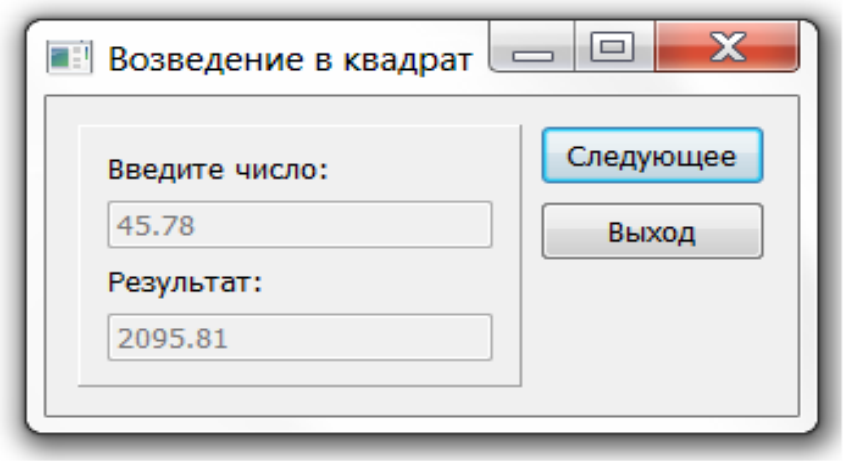




Файл win.cpp. Создание и настройка виджетов

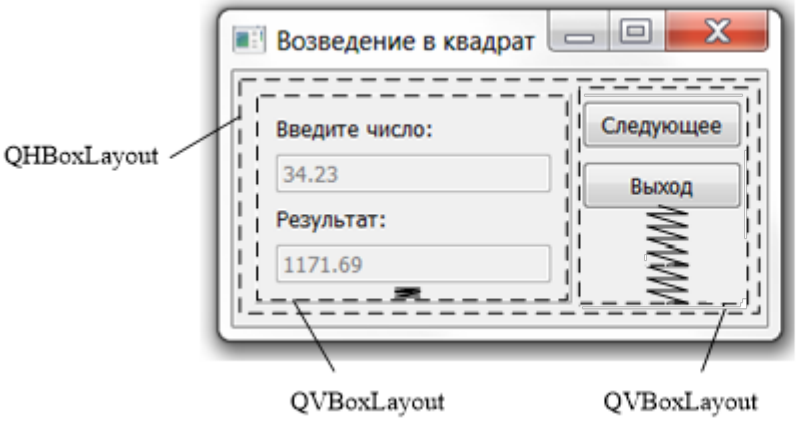
```
outputLabel =
      new QLabel(codec->toUnicode("Результат:"),this);
outputEdit = new QLineEdit("",this);
nextButton =
      new QPushButton(codec->toUnicode("Следующее"),this);
exitButton =
      new QPushButton(codec->toUnicode("Выход"),this);
```





Файл win.cpp. Компоновка виджетов

```
QVBoxLayout *vLayout1 = new QVBoxLayout(frame);
vLayout1->addWidget(inputLabel);
vLayout1->addWidget(inputEdit);
vLayout1->addWidget(outputLabel);
vLayout1->addWidget(outputEdit);
vLayout1->addStretch();
```



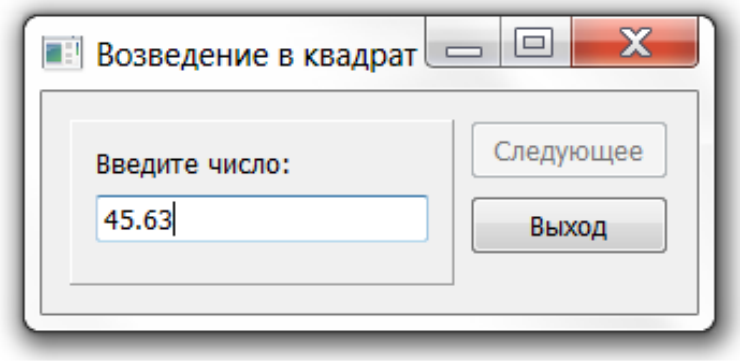

```
QVBoxLayout *vLayout2 = new QVBoxLayout();
vLayout2->addWidget(nextButton);
vLayout2->addWidget(exitButton);
vLayout2->addStretch();

QHBoxLayout *hLayout = new QHBoxLayout(this);
hLayout->addWidget(frame);
hLayout->addLayout(vLayout2);
```

Файл win.cpp. Инициализация интерфейса и связывание сигналов и слотов

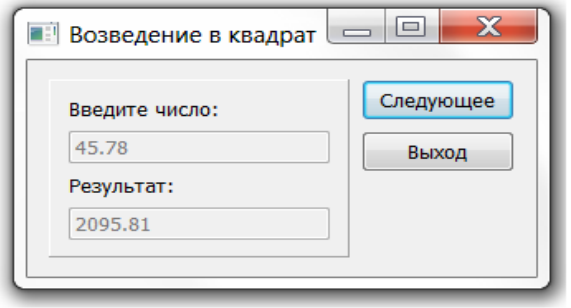
```
    begin();     // инициализация интерфейса

    // связь нажатия кнопки Выход и закрытия главного окна
    connect(exitButton,SIGNAL(clicked(bool)),
            this,SLOT(close()));
    // связь нажатия кнопки Следующее и инициализации интерфейса
    connect(nextButton,SIGNAL(clicked(bool)),
            this,SLOT(begin()));
    // связь нажатия клавиши Enter и вычислений
    connect(inputEdit,SIGNAL(returnPressed()),
            this,SLOT(calc()));
}
```

Файл win.cpp. Метод начальной настройки (инициализации) интерфейса

```
void Win::begin()
{
    inputEdit->clear();                    // очистка строки ввода
    nextButton->setEnabled(false);         // деактивация кнопки Следующее
    nextButton->setDefault(false);         // отмена активации кнопки
                                           // Следующее при нажатии Enter
    inputEdit->setEnabled(true);           // активация строки ввода
    outputLabel->setVisible(false);        // сокрытие метки результата
    outputEdit->setVisible(false);         // сокрытие строки результата
    inputEdit->setFocus();                 // установка фокуса на строку ввода
}
```



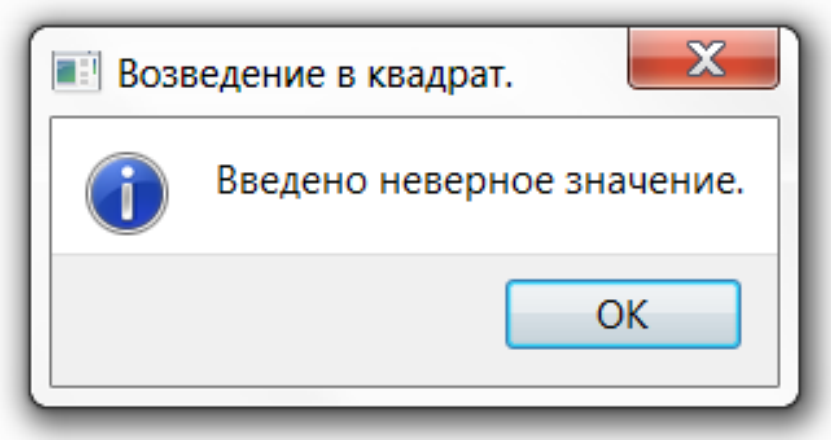
Файл win.cpp. Вычисление результата




```
void Win::calc()
{
    bool Ok=true;
    float r,a;
    QString str=inputEdit->text();  // копирование введенной строки
    a=str.toDouble(&Ok);            // преобразование строки в число
    if (Ok)                         // если преобразование успешно, то
    {
        r=a*a;                      // возводим число в квадрат
        str.setNum(r);              // преобразуем число в строку
        outputEdit->setText(str);   // заносим результат в окно результата
        inputEdit->setEnabled(false);  // деактивируем окно ввода
        outputEdit->setEnabled(false);  // деактивируем окно вывода
        outputLabel->setVisible(true); // показываем метку результата
        outputEdit->setVisible(true);  // показываем окно результата
        nextButton->setDefault(true);  // назначаем Следующее кнопкой
                                       // активируемой по нажатию клавиши Enter
        nextButton->setFocus();        // устанавливаем фокус и активируем
        nextButton->setEnabled(false); // кнопку Следующее
    }
```

Файл win.cpp. Метод вычислений: выдача сообщения об ошибке ввода

```
else
        if (!str.isEmpty())
        {
            QMessageBox msgBox(QMessageBox::Information,
                codec->toUnicode("Возведение в квадрат."),
                codec->toUnicode("Введено неверное значение."),
                QMessageBox::Ok);
            msgBox.exec();
        }
}
```



Файл win.cpp. Метод проверки строки

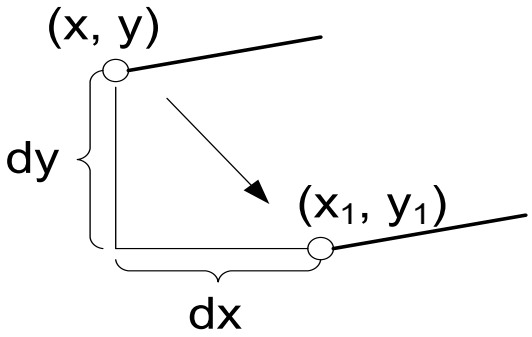
```
QValidator::State
        StrValidator::validate(QString &str,int &pos)const
{
    return Acceptable;  // метод всегда принимает
                        // вводимую строку
}
```

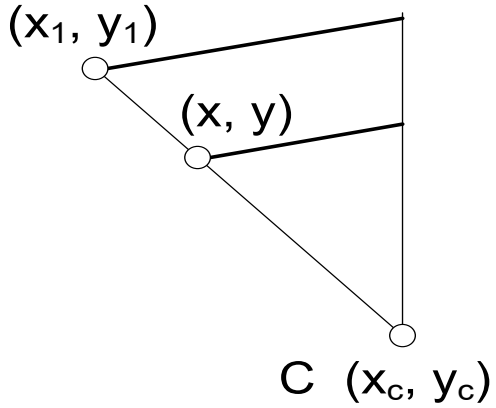
Файл main.cpp

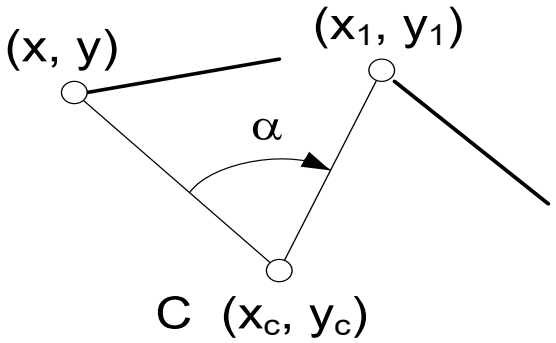
```
#include "win.h"
#include <QApplication>

int main(int argc, char *argv[])
{
   QApplication app(argc, argv);
   Win win;
   win.show();
   return app.exec();
}
```

9.6 Создание приложений с движущимися изображениями

Три составляющих движения объекта на плоскости:







1 Перемещение

$x1 = x + dx,$

$y1 = y + dy$

2 Масштабирование относительно точки C (xc, yc):

$x1 = (x - xc)*M_x + xc,$

$y1 = (y - yc)*M_y + yc$

3 Поворот относительно точки C(xc, yc):

$x1 = (x - xc)\cos\alpha + (y - yc)\sin\alpha + xc,$

$y1 = (y - yc)\cos\alpha - (x - xc)\sin\alpha + yc$

Обработка событий. Рисование. Таймеры

Пример 9.4 Создание движущихся изображений

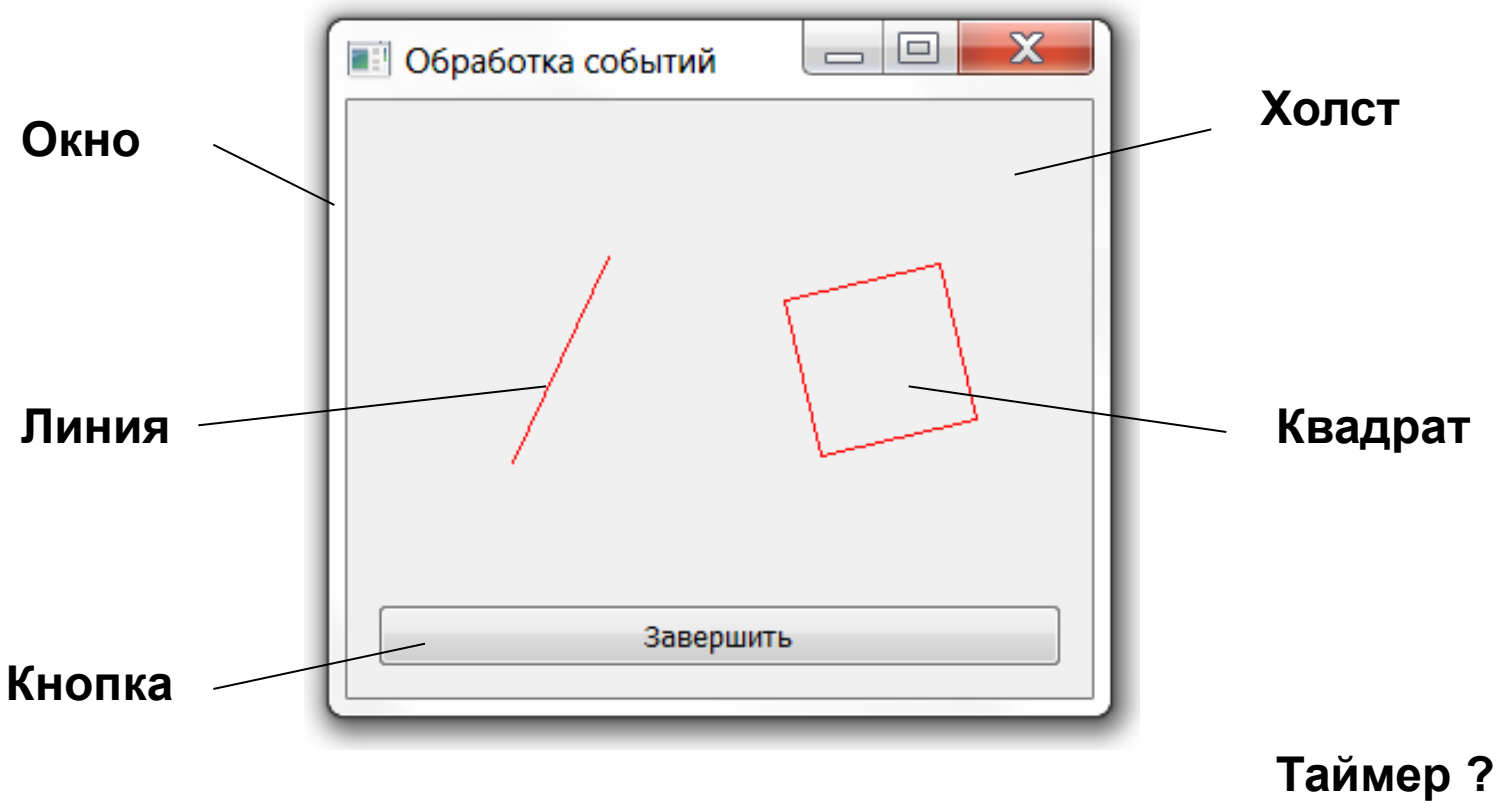


Диаграмма объектов приложения

Кроме объекта Приложение программа включает 6 объектов:

- **окно;**
- **кнопка Выход;**
- **холст для рисования;**
- **таймер;**
- **линия;**
- **квадрат.**

Диаграмма объектов

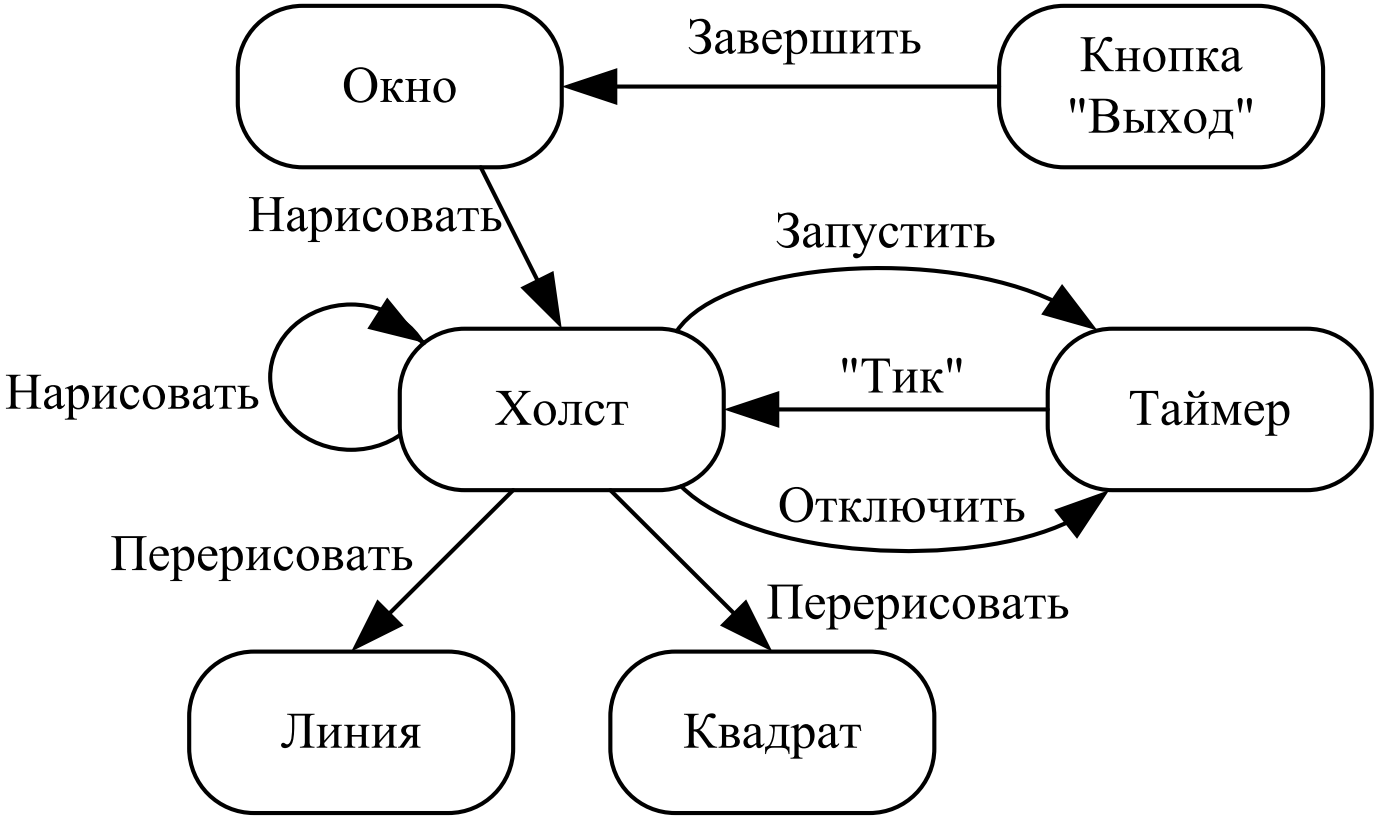
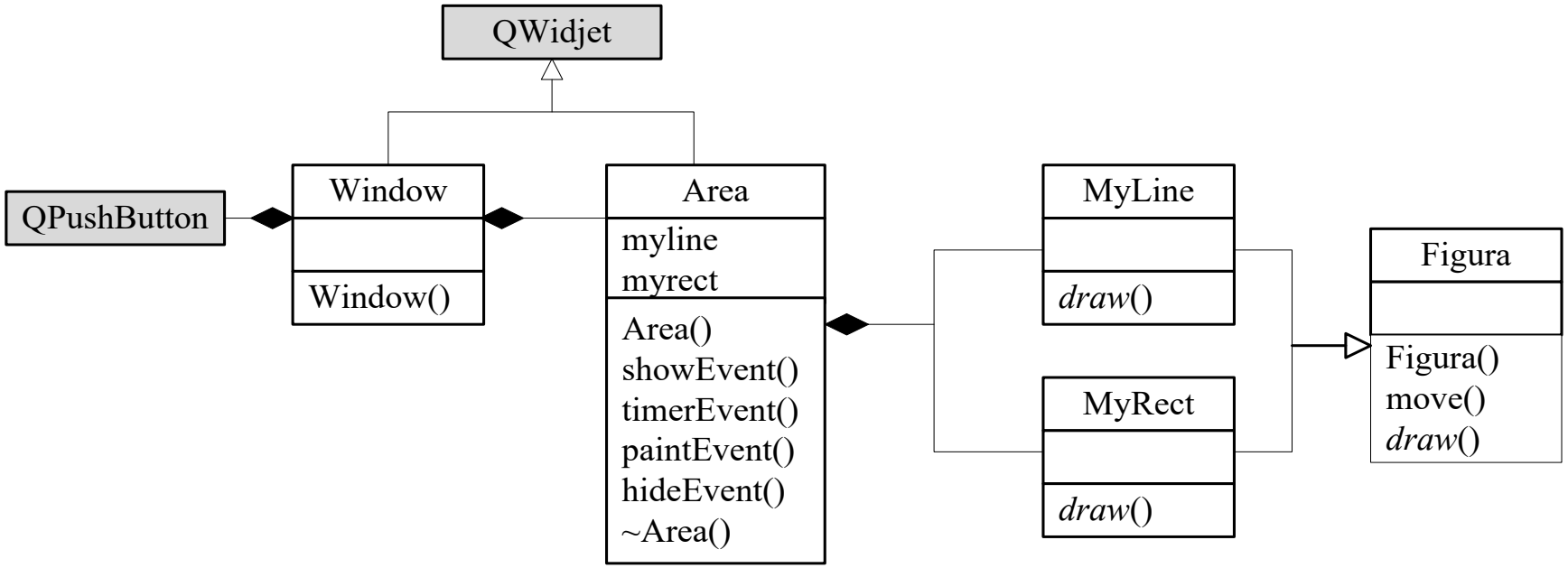


Диаграмма классов приложения



События:

- showEvent() – при визуализации области рисования – включаем таймер;
- timerEvent() – при получении сообщения от таймера – инициируем перерисовку окна;
- paintEvent() – при получении запроса на перерисовку области рисования – перерисовываем фигуры;
- hideEvent() – при сокрытии области рисования – выключаем таймер.

Диаграмма последовательностей действий

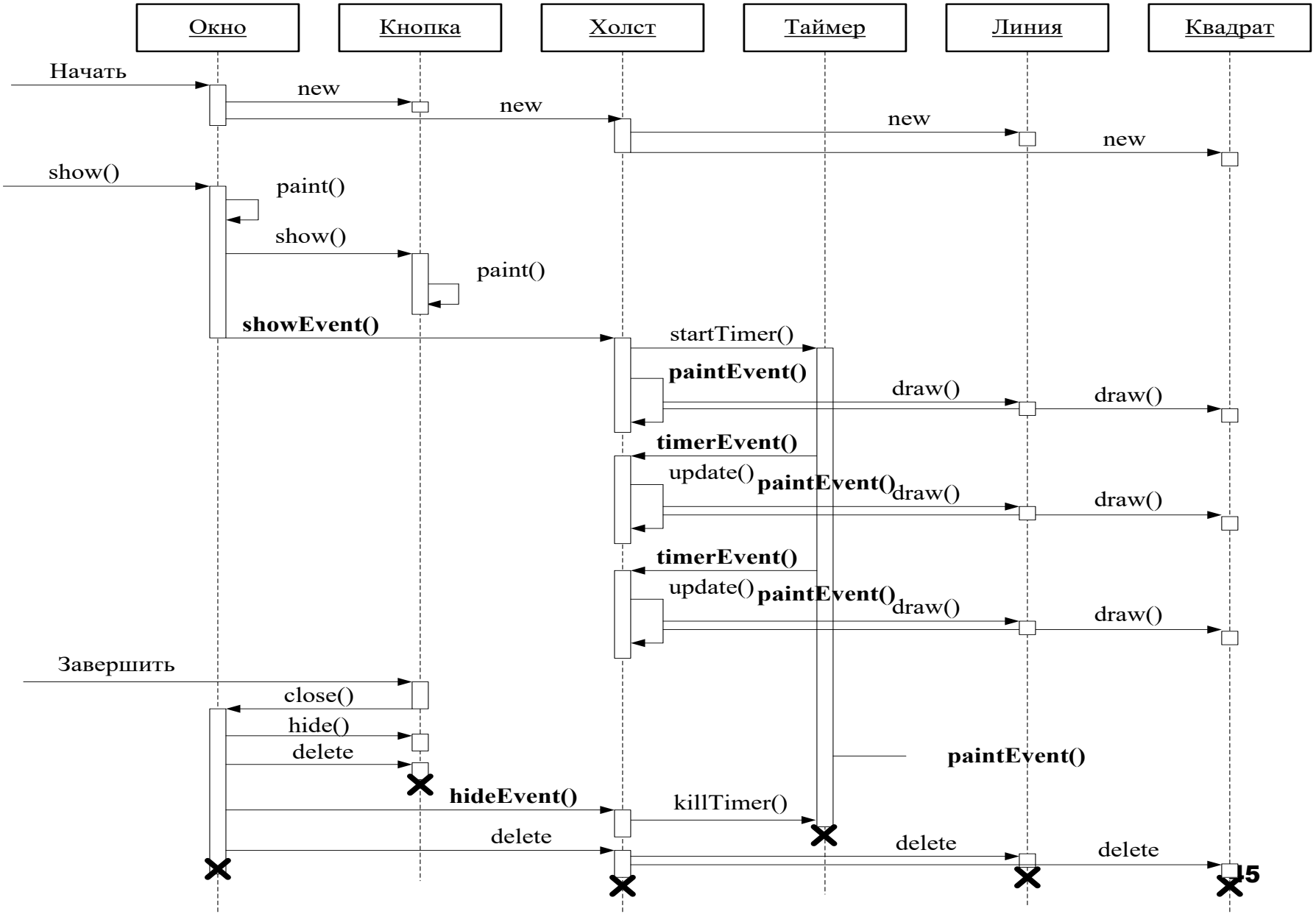
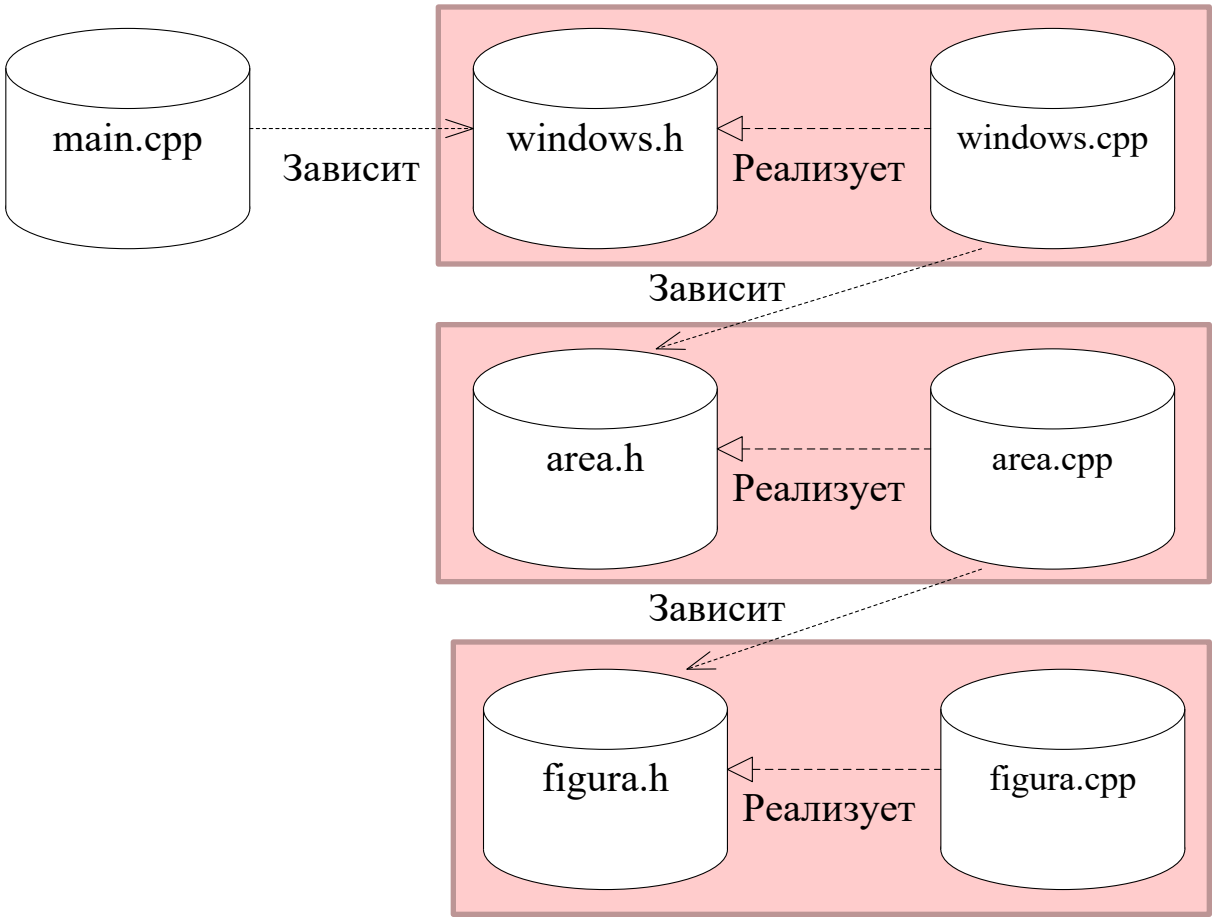


Диаграмма компоновки приложения

Модуль Окно содержит описание класса окна приложения.

Модуль Область рисования содержит описание класса Холст.

Модуль Фигура содержит описание классов Фигура, Линия и квадрат.



Файл figura.h. Класс Фигура:

```
#ifndef figura_h
#define figura_h
#include <QPainter>
class Figura
{
protected:
    int x,y,halflen,dx,dy,r;
    virtual void draw(QPainter *Painter)=0;
public:
    Figura(int X,int Y,int Halflen):
                                x(X),y(Y),halflen(Halflen){}
    void move(float Alpha,QPainter *Painter);
    virtual ~Figura(){} // необходим для полиморфного объекта
};
```

Файл figura.h. Классы Линия и Квадрат:

```
class MyLine:public Figura
{
protected:
    void draw(QPainter *Painter);
public:
    MyLine(int x,int y,int halflen):Figura(x,y,halflen){}
};
class MyRect:public Figura
{
protected:
    void draw(QPainter *Painter);
public:
    MyRect(int x,int y,int halflen):Figura(x,y,halflen){}
};
#endif
```

Файл figura.cpp

```
#include "figura.h"
#include <math.h>
void Figura::move(float Alpha,QPainter *Painter)
{
    dx=static_cast<int>(halflen*cos(Alpha));
    dy=static_cast<int>(halflen*sin(Alpha));
    draw(Painter);    // необходим сложный полиморфизм!
}
void MyLine::draw(QPainter *Painter)
{
    Painter->drawLine(x+dx,y+dy,x-dx,y-dy);
}
void MyRect::draw(QPainter *Painter)
{
    Painter->drawLine(x+dx,y+dy,x+dy,y-dx);
    Painter->drawLine(x+dy,y-dx,x-dx,y-dy);
    Painter->drawLine(x-dx,y-dy,x-dy,y+dx);
    Painter->drawLine(x-dy,y+dx,x+dx,y+dy);
}
```

Файл area.h

```
#ifndef AREA_H
#define AREA_H
#include "figura.h"
#include <QWidget>
#include <QTimer>
class Area : public QWidget
{
    int myTimer; // идентификатор таймера
    float alpha; // угол поворота
public:
    Area(QWidget *parent = nullptr);
    ~Area();
    MyLine *myline;    // указатели на объекты фигур
    MyRect *myrect;
protected:
    //   обработчики событий
    void paintEvent(QPaintEvent *event);
    void timerEvent(QTimerEvent *event);
    void showEvent(QShowEvent *event);
    void hideEvent(QHideEvent *event);
};
#endif
```

Файл area.cpp. Обработка событий визуализации и перерисовки

```
#include "area.h"
#include <QTimerEvent>
Area::Area(QWidget *parent):QWidget(parent)
{
   setFixedSize(QSize(300,200)); // фиксируем размер Холста
   myline=new MyLine(80,100,50);  // создаем объект Линия
   myrect=new MyRect(220,100,50); // создаем объект Квадрат
   alpha=0;
}
void Area::showEvent(QShowEvent *)
{
    myTimer=startTimer(50);        // включаем таймер
}
void Area::paintEvent(QPaintEvent *)
{
    QPainter painter(this);      // создаем контент рисования на Холсте
    painter.setPen(Qt::red);     // задаем красное Перо
    myline->move(alpha,&painter);            // рисуем Линию
    myrect->move(alpha*(-0.5),&painter);  // рисуем Квадрат
}
```

Файл area.cpp. Обработка событий таймера и сокрытия Холста

```
void Area::timerEvent(QTimerEvent *event)
{
    if (event->timerId() == myTimer)  // если наш таймер
    {
        alpha += 0.2;
        update();            // обновить внешний вид
    }
    else
        QWidget::timerEvent(event);   // иначе событие передать далее
}
void Area::hideEvent(QHideEvent *)
{
   killTimer(myTimer);   // выключить таймер
}
Area::~Area()
{
    delete myline;
    delete myrect;
}
```

Файл window.h

```
#ifndef window_h
#define window_h
#include "area.h"
#include <QWidget>
#include <QTextCodec>
#include <QPushButton>

class Window : public QWidget
{
protected:
    QTextCodec *codec;
    Area * area;            // область отображения рисунка
    QPushButton * btn;
public:
    Window();
};
#endif
```

Файл window.cpp

```
#include "window.h"
#include <QVBoxLayout>

Window::Window()
{
    codec = QTextCodec::codecForName("Windows-1251");
    this->setWindowTitle(codec->toUnicode("Обработка событий"));
    area = new Area( this );
    btn = new QPushButton(codec->toUnicode("Завершить"),this );
    QVBoxLayout *layout = new QVBoxLayout(this);
    layout->addWidget(area);
    layout->addWidget(btn);
    connect(btn, SIGNAL(clicked(bool)),this,SLOT(close()));
};
```

Файл main.cpp

```
#include "window.h"
#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication appl(argc, argv);
    Window win;
    win.show();
    return appl.exec();
}
```